CIRCUITICIAN™
BE CREATIVE

MBEDX

ESP32 Embedded Systems Workshop

From Registers to IoT — Learn. Build. Deploy.

2 Days • 10 Hours • 12 Modules • PlatformIO + ESP-IDF • 11:00 AM – 5:00 PM

DAY 1

Foundations & Sensing

01

GPIO • Interrupts • ADC - DAC &PWM • EEPROM

8	5	18+	2
Modules	Hours	Exercises	Bare Metal Codes

Day 1 — Schedule at a Glance

Foundations & Sensing • 11:00 AM – 5:00 PM

11:00 – 11:20	**Welcome & Kit Setup**	*Icebreaker, team formation, PlatformIO check*
11:20 – 12:20	**Module 1 — Getting Started**	*ESP32 architecture, GPIO, PlatformIO, first code*
12:20 – 13:00	**Module 2 — Digital I/O & Interrupts**	*Polling vs ISR, IR sensor, Tachometer*
13:00 – 13:45	**Module 3 — Bare Metal & Registers**	*ESP-IDF, datasheet reading, register programming*
13:45 – 14:30	Lunch Break	
14:30 – 15:15	**Module 4 — Analog Sensing (ADC)**	*LDR, voltage divider, 12-bit ADC, light zones*
15:15 – 15:40	**Module 5 — PWM & Output Control**	*LED brightness, LEDC API, tone generation*
15:40 – 16:00	**Module 6 — DAC & Output Control**	*DAC Resolution, wave generation, Audio*
16:00 – 16:30	**Module 7 — DHT11 & Protocols**	*Single-wire protocol, temperature alerts*
16:30 – 16:45	**Module 8 — EEPROM**	*Save your data permanently*
16:45 – 17:00	**Day 1 Recap & Quiz**	*5 rapid-fire questions + Day 2 preview*

OPENING

What is an Embedded System?

The computers hiding inside everything around you

ABS Brakes

Reads wheel sensor 1000×/sec

Smartphone

10+ microcontrollers inside

Traffic Light

State machine + timers

Washing Machine

Sensor fusion + motor control

Smartwatch

BLE + sensors + power mgmt

Smart Thermostat

Exactly what you'll build today!

Today's goal: Understand every layer — from silicon registers to Wi-Fi web servers.

DAY 1

MODULE 1

Getting Started with ESP32

Architecture • GPIO • PlatformIO • Your First Program

MODULE 1

ESP32 Architecture

Why this chip dominates the IoT and embedded world

Dual-Core Xtensa LX6

240 MHz — Core 0: OS/Wi-Fi Stack | Core 1: Your Code

Memory

520 KB SRAM • 4 MB Flash • 8 KB RTC RAM (survives deep sleep)

Connectivity

Wi-Fi 802.11 b/g/n • Bluetooth 4.2 & BLE — both built-in

Peripherals

34 GPIO • 12-bit ADC • DAC • SPI / I2C / UART / I2S

ESP32 vs Arduino UNO

Feature	ESP32	UNO
CPU	240 MHz	16 MHz
RAM	520 KB	2 KB
Flash	4 MB	32 KB
Wi-Fi	✓ Built-in	✗ None
ADC	12-bit	10-bit
Price	~₹260	~₹440

⚠ 3.3V ONLY — GPIO pins are NOT 5V tolerant. Connecting 5V directly causes permanent damage.

MODULE 1

GPIO Pinout & PlatformIO Setup

GPIO Pin Categories

GPIO 0,2,4,5,12–19,21–23,25–27,32–33
General Purpose — Input & Output

GPIO 34, 35, 36, 39
Input ONLY — no output capability

GPIO 0, 2, 5, 12, 15
Strapping pins — affect boot mode

GPIO 32–39 (ADC1)
ADC safe even when Wi-Fi is ON ✓

GPIO 4,0,2,15,13,12,14,27,33,32
Capacitive Touch Pins T0–T9

platformio.ini — Project Config

```
[env:esp32dev]
platform  = espressif32
board     = esp32dev
framework = arduino
monitor_speed = 115200

; Add libraries here:
lib_deps =
  adafruit/DHT sensor library
```

`LED Resistor = (3.3V – 2.0V) ÷ 0.010A = 130Ω → use 220Ω`

Exercises:

1.1 Serial Hello
1.2 LED Blink
1.3 Button INPUT_PULLUP
1.4 Button Toggles LED

DAY 1

MODULE 2

Digital I/O & Interrupts

Polling vs ISR • IR Sensor • Hardware-Driven Response

MODULE 2

Polling vs Hardware Interrupts

The most important concept in real-time embedded systems

✕ Polling — The Wrong Way

```
void loop() {
  // CPU stuck here:
  while(digitalRead(IR)==HIGH){
    // doing NOTHING else
  }
  triggerAlert();
}
```

CPU burns 100% checking a pin that rarely changes.
Cannot read sensors, serve web pages, or do anything else simultaneously.

✓ Interrupt — The Right Way

```
volatile bool flag = false;

void IRAM_ATTR onIR() {
  flag = true; // short ISR!
}
void setup() {
  attachInterrupt(IR, onIR, FALLING);
}
```

CPU is free to do other work. Hardware calls the ISR instantly when event occurs — no polling needed.

ISR Rules: IRAM_ATTR • Keep it SHORT • No Serial.print() • No delay() • volatile for shared vars

MODULE 2 — EXERCISES

Hands-On Exercises

Ex 2.1

IR Sensor — Polling First

Wire IR to GPIO 14. Poll in loop, print Object/Clear. Feel how it blocks everything.

Ex 2.2

Interrupt-Driven IR + LED

Rewrite using attachInterrupt(). volatile bool flag. LED ON for 2s on detection.

Mini Project 2.3 — RPM Counter

1 → LED blinks at 500ms interval

2 → Buzzer beeps if RPM>1000

3 → Serial: ‘RPM:X'

DAY 1

MODULE 3

Bare Metal & Register Programming

3

ESP-IDF • Datasheet Reading • Direct Hardware Control

MODULE 3

The Abstraction Stack

Every Arduino function is a wrapper — today you go under the hood

YOUR CODE

```
digitalWrite(GPIO_NUM_2, HIGH);
```

What you write in Arduino / PlatformIO

↓

ESP-IDF HAL

```
gpio_set_level(GPIO_NUM_2, 1);
```

Hardware Abstraction Layer by Espressif

↓

REGISTER OPS

```
GPIO_OUT_REG |= (1 << 2);
```

Direct memory-mapped I/O — what you write TODAY

↓

SILICON

```
Transistor gate → 3.3V on pin
```

Physics — voltage on the copper trace

Why learn this? Safety-critical systems (automotive, medical, aerospace) require understanding every layer beneath your code.

MODULE 3 — BARE METAL

Datasheet Reading & Register Exercises

Open the ESP32 Technical Reference Manual — Chapter 4: GPIO

Key GPIO Registers (from TRM Chapter 4)

GPIO_OUT_REG	0x3FF44004	Set output value GPIO 0–31
GPIO_ENABLE_REG	0x3FF44020	Enable GPIO 0–31 as output
GPIO_IN_REG	0x3FF44038	Read input state GPIO 0–31
GPIO_OUT_W1TS_REG	0x3FF44008	Write 1 → SET specific bits
GPIO_OUT_W1TC_REG	0x3FF4400C	Write 1 → CLEAR specific bits

🕮 Your Datasheet Task — Find these answers in TRM

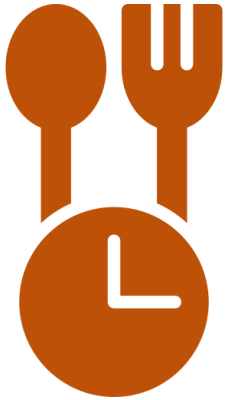
1. Address of GPIO_OUT_REG?
2. Which bit position controls GPIO2?
3. How to enable GPIO2 as output?
4. Address & bit for GPIO15 input?

Ex 3.1 + 3.2 — Fill the Blanks

```
// Ex 3.1 — LED Blink (ESP-IDF)
#define GPIO_OUT_REG  \
 (*(volatile uint32_t*)0x3FF44004)
#define GPIO_ENABLE_REG \
 (*(volatile uint32_t*)0x3FF44020)

void app_main(void) {
  // TODO: Enable GPIO2 output
  GPIO_ENABLE_REG |= _____;
  while(1) {
    GPIO_OUT_REG |= _____;  // HIGH
    vTaskDelay(500/portTICK_MS);
    GPIO_OUT_REG &= _____;  // LOW
  }
}

// Ex 3.2 — Button Read (GPIO_IN_REG)
// Read bit 15 WITHOUT digitalRead()
if (!((GPIO_IN_REG >> ___) & 1)) {
  // button pressed — toggle LED
}
```



LUNCH

45 minutes • Back at 14:30 sharp

DAY 1

MODULE 4

Analog Sensing & ADC

LDR • Voltage Divider • 12-bit Precision • Light Zones

MODULE 4

Analog-to-Digital Conversion (ADC)

The real world is analog — ESP32's 12-bit ADC bridges the gap

12-Bit Resolution

0–3.3V in 4096 steps = 0.8mV per step
Arduino UNO = 10-bit (1024 steps — 4× less precise)

ADC1 vs ADC2 ⚠

ADC2 conflicts with Wi-Fi — ALWAYS use ADC1
Safe pins: GPIO 32–39 (ADC1 channels only)

Voltage Divider

LDR + 10kΩ = divider. Bright → LDR low → V rises
Vout = Vsupply × R_fixed ÷ (R_LDR + R_fixed)

Exercises

4.1 Raw LDR + Serial Plotter

GPIO34. Print raw. Use PlatformIO Plotter for live graph.

4.2 Map to Light %

map(raw,0,4095,0,100). Print 'Light Level: 73%'

4.3 Mini: Smart Street Light

BRIGHT>70%: LEDs off | DIM 30–70%: 1 LED | DARK<30%: all LEDs + buzzer on state change

```
int raw = analogRead(34);                          // 0 – 4095
int pct = map(raw, 0, 4095, 0, 100);               // 0 – 100%
Serial.printf("Light: %d%% | Raw: %d\n", pct, raw); // print both
```

DAY 1

MODULE 5

PWM & Output Control

5

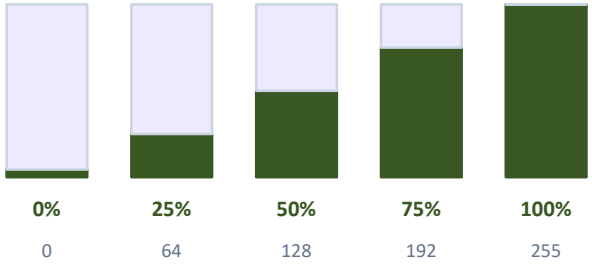
LEDC API • LED Dimming • Tone Generation • Duty Cycle

MODULE 5

PWM — Pulse Width Modulation

Simulate analog output with digital switching — hardware-accurate, zero CPU overhead

Duty Cycle = ON_time ÷ Period × 100%



ESP32 LEDC API (PlatformIO / Arduino)

```
ledcSetup(0, 5000, 8);  // ch, freq Hz, bits
ledcAttachPin(2, 0);    // GPIO, channel
ledcWrite(0, 128);      // ch, duty (0-255)

// For buzzer tones (same API!):
ledcWriteTone(1, 523);  // ch, frequency Hz
```

Exercises

5.1 Smooth LED Fade

0→255→0 loop, breathing effect

5.2 Button Brightness Steps

0%→25%→50%→75%→100%→0% each press

5.3 Tone Scale

523Hz C5, 659Hz E5, 784Hz G5 — boot jingle

5.4 Mini: LDR Auto-Dimmer

Bright env = dim LED. Dark = bright. Continuous, no thresholds.

DAY 1

MODULE 6

DAC & Output Control

6

Tune Analog Output • 8-bit Resolution • Sine Waves • Audio

MODULE 6

DAC — Digital to Analog Converter

ESP32's built-in DAC produces TRUE analog voltage — no switching, no PWM approximation

ESP32 DAC — Key Facts

Channels:	2 independent DACs — DAC1 & DAC2
Pins:	DAC1 = GPIO 25 | DAC2 = GPIO 26
Resolution:	8-bit → 256 steps (0 = 0V, 255 = 3.3V)
Step Size:	3.3V ÷ 255 ≈ 12.9 mV per step

dacWrite() API — Simple & Direct

```
// Basic usage — one line!
dacWrite(25, 128);   // GPIO25 → 1.65V
dacWrite(26, 0);     // GPIO26 → 0.0V
dacWrite(25, 255);   // GPIO25 → 3.3V

// Sweep (simple analog ramp):
for (int v=0; v<=255; v++) {
  dacWrite(25, v);
  delayMicroseconds(500);
}
```

DAC vs PWM — When to Use Which?

Property	DAC ✓	PWM
Output Type	TRUE analog voltage	Digital pulses (fake analog)
Smoothness	Perfectly smooth signal	Has ripple / switching noise
Best for	Audio, sine waves, sensors	LED dim, motor speed, buzzers
CPU Load	Zero — hardware peripheral	Zero — LEDC hardware timer
Resolution	8-bit (256 steps)	Up to 16-bit (65536 steps)

Advanced: Sine Wave Generation

```
#include <math.h>
// Generate 1Hz sine wave centered at 1.65V
void loop() {
  for (int i=0; i<360; i++) {
    float rad = i * (PI / 180.0);
    int val = 128 + (int)(127*sin(rad));
    dacWrite(25, val); // 0-255
    delayMicroseconds(2778); // ~1Hz
  }
}
```

DAY 1

MODULE 7

DHT11 & Sensor Protocols

Single-Wire • Temperature • Humidity • Error Handling

MODULE 7

DHT11 — Sensor Communication Protocols

Why digitalRead() can't decode a DHT11 — and how libraries solve it

Protocol Landscape

UART
Async serial, 2 wires

SPI
High-speed, 4 wires

I2C
2 wires, multi-device

1-Wire
1 data wire, slow

DHT11 ← YOU ARE HERE
Custom 1-wire, 40-bit, ~70µs timing

DHT11 Specs

Temp Range:	**0–50°C (±2°C)**
Humidity:	**20–80% (±5%)**
Sample Rate:	**Max 1 reading / second**
Data Frame:	**40 bits: Humi + Temp + Checksum**

```
// platformio.ini → lib_deps = adafruit/DHT sensor library
#include <DHT.h>
DHT dht(4, DHT11);   // GPIO4
dht.begin();
float t = dht.readTemperature();
float h = dht.readHumidity();
if (isnan(t) || isnan(h)) { /* handle error! */ }
float hi = dht.computeHeatIndex(t, h, false);
```

Ex 7.1 — Read + display Temp/Humidity/HeatIndex every 2s
Ex 7.2 — Mini Project: 3-zone alert (<20°C BLUE | 20-35°C GREEN | >35°C RED + Buzzer alarm)

DAY 1

MODULE 8

EEPROM — Persistent Memory

Non-Volatile Storage • Wear Leveling • Key-Value Data

MODULE 7

EEPROM — Saving Data That Survives Power Loss

Write once, read forever — persists across resets, deep sleep, and power cuts

What is EEPROM on ESP32?

Not Real EEPROM

ESP32 has no physical EEPROM chip — the library emulates it using a 4 KB region of SPI Flash.

512 Bytes Default

EEPROM.begin(512) — allocates up to 512 bytes.
Each address stores one byte (0–255).

Wear Leveling

Flash cells die after ~100,000 write cycles.
Avoid writing in tight loops — batch your writes!

commit() is Key

EEPROM.write() only changes the RAM buffer.
EEPROM.commit() physically saves to flash.

EEPROM vs Preferences library: For larger data or key-value pairs, use Preferences.h (NVS) instead — more wear-resistant and easier to use.

EEPROM API — Read & Write

```
#include <EEPROM.h>
EEPROM.begin(512); // allocate 512 bytes

// ── WRITE ──────────────────────────
EEPROM.write(0, 42);   // addr 0 = 42
EEPROM.put(4, 3.14f);  // float at addr 4
EEPROM.commit();       // MUST call this!

// ── READ ───────────────────────────
int val = EEPROM.read(0);   // → 42
float f;
EEPROM.get(4, f);           // → 3.14
```

Exercises

- **8.1 Boot Counter —** Count reboots, store in EEPROM addr 0. Print on start.
- **8.2 Save LED State —** Button toggles LED. Save ON/OFF to EEPROM — persists after reset.
- **8.3 Sensor data Store —** Write name + threshold float. Read back and print on boot.

Day 1 — Rapid Fire Quiz

Teams answer on paper — fastest correct team wins 📝

Q1	What is the memory address of GPIO_OUT_REG on ESP32?
Q2	Which ADC channel group must you avoid when using Wi-Fi?
Q3	What does the volatile keyword mean in the context of an ISR?
Q4	PWM is ON for 2ms in a 10ms period — what is the duty cycle?
Q5	Name ONE reason why delay() is dangerous in real embedded systems.

🚀 **Tomorrow: Your ESP32 connects to Wi-Fi. Your phone becomes a remote control.**